

EXPERIMENT 5

Objective: To make Linear Queue, Circular Queue and Priority Queue using Linked List operations.

Linear Queue

```
#include <bits/stdc++.h>
using namespace std;

class Node {
public:
    int data;
    Node* next;
    Node* prev;
    Node(int value) {
        data = value;
        next = nullptr;
        prev = nullptr;
    }
};

void display(Node* head) {
    Node* temp = head;
    if(temp == nullptr) {
        cout << "Queue is empty" << endl;
        return;
    }

    while(temp != nullptr) {
        cout << temp->data << " ";
        temp = temp->next;
    }
    cout << endl;
}

void push(Node*& head, Node*& rear, int value) {
```

```

Node* newNode = new Node(value);
if(rear == nullptr) {
    head = newNode;
    rear = newNode;
}
else {
    rear->next = newNode;
    newNode->prev = rear;
    rear = newNode;
}
}

```

```

void pop(Node*& head, Node*& rear) {
    if(head == nullptr) {
        cout << "Queue is empty" << endl;
        return;
    }
    else {
        Node* temp = head;
        head = head->next;
        if(head != nullptr) {
            head->prev = nullptr;
        }
        else {
            rear = nullptr;
        }
    }
}

```

```

int main() {
    int n;

    cout << "Enter the number of elements to be pushed into the queue: ";
    cin >> n;

    Node* head = nullptr;
    Node* rear = nullptr;

    cout << "Enter the elements: ";

    for (int i = 0; i < n; i++) {

```

```

        int x;
        cin >> x;
        push(head, rear, x);
    }

    cout << "Linear Queue: ";
    display(head);

    pop(head, rear);
    display(head);

    return 0;
}

```

Output:

```

PS C:\Second Year\3rd Semester\Advanced DSA using C++\Queue with Linked List> cd "c:\Second Year\3rd Semester\Advanced DSA using C++\Queue with Linked List\" ; if ($?) { g++ Linear_queue.cpp -o Linear_queue } ; if ($?) { .\Linear_queue.exe }
Enter the number of elements to be pushed into the queue: 6
Enter the elements: 44 525 5 22 567 24
Linear Queue: 44 525 5 22 567 24
525 5 22 567 24
PS C:\Second Year\3rd Semester\Advanced DSA using C++\Queue with Linked List>

```

Circular Queue

```

#include <bits/stdc++.h>
using namespace std;

```

```

class Node {
public:
    int data;
    Node* next;
    Node* prev;
    Node(int value) {
        data = value;
        next = nullptr;
        prev = nullptr;
    }
};

```

```

void display(Node* head) {

```

```

Node* temp = head;
if(temp == nullptr) {
    cout << "Queue is empty" << endl;
    return;
}

do {
    cout << temp->data << " ";
    temp = temp->next;
}
while(temp != head);
}

void push(Node*& head, Node*& rear, int value) {
    Node* newNode = new Node(value);
    if(rear == nullptr) {
        head = newNode;
        rear = newNode;
        newNode->next = newNode;
        newNode->prev = newNode;
    }
    else {
        newNode->prev = rear;
        newNode->next = head;

        rear->next = newNode;
        head->prev = newNode;

        rear = newNode;
    }
}

void pop(Node*& head, Node*& rear) {
    if(head == nullptr) {
        cout << "Circular Queue is empty" << endl;
        return;
    }
    else if(head == rear) {
        delete head;
        head = nullptr;
        rear = nullptr;
    }
}

```

```

    }
    else {
        Node* temp = head;
        head = head->next;
        head->prev = rear;
        rear->next = head;
        delete temp;
    }
}

int main() {
    int n;
    cout << "Enter the number of elements to be pushed into the circular queue: ";
    cin >> n;

    Node* head = nullptr;
    Node* rear = nullptr;

    cout << "Enter the elements: ";
    for (int i=0; i<n; i++) {
        int x;
        cin >> x;
        push(head, rear, x);
    }

    cout << "Circular queue: ";
    display(head);

    pop(head, rear);
    cout << "\nCircular queue after popping an element: ";
    display(head);
}

```

Output:

```

PS C:\Second Year\3rd Semester\Advanced DSA using C++\Queue with Linked List> cd "c:\Second Year\3rd Semester\Advanced DSA using C++\Queue with Linked List\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) {
Enter the number of elements to be pushed into the circular queue: 6
Enter the elements: 4535 55 3354 67 57 7
Circular queue: 4535 55 3354 67 57 7
Circular queue after popping an element: 55 3354 67 57 7
PS C:\Second Year\3rd Semester\Advanced DSA using C++\Queue with Linked List>

```

Priority Queue

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
class Node{
```

```
    public:
```

```
    int data;
```

```
    Node* next;
```

```
    Node* prev;
```

```
    Node(int value) {
```

```
        data = value;
```

```
        next = nullptr;
```

```
        prev = nullptr;
```

```
    }
```

```
};
```

```
void display(Node* head){
```

```
    if(head == nullptr){
```

```
        cout << "Priority queue is empty" << endl;
```

```
        return;
```

```
    }
```

```
    Node* temp = head;
```

```
    while(temp != nullptr) {
```

```
        cout << temp->data << " ";
```

```
        temp = temp->next;
```

```
    }
```

```
    cout<<endl;
```

```
}
```

```
void push(Node*& head, int value) {
```

```
    Node* temp = head;
```

```
    Node* newnode = new Node(value);
```

```
    if(head == nullptr || head->data > value){
```

```
        newnode->next = head;
```

```
        head = newnode;
```

```
        return;
```

```
    }
```

```
    while(temp->next != nullptr && temp->next->data <= value){
```

```
        temp = temp->next;
```

```
    }
```

```

    newnode->next = temp->next;
    temp->next = newnode;
}

void pop(Node*& head){
    if(head == nullptr){
        cout<<"Priority queue is empty"<<endl;
        return;
    }
    if(head->next == nullptr){
        head = head->next;
        cout << "Priority queue is empty" << endl;
        return;
    }
    head = head->next;
}

int main() {
    int n;
    cout << "Enter the number of elements to be inserted in the priority queue: ";
    cin >> n;

    Node* head = nullptr;
    Node* rear = nullptr;

    cout << "Enter the elements: ";
    for (int i=0; i<n; i++) {
        int x;
        cin >> x;
        push(head, x);
    }

    cout << "Elements in the priority queue: ";
    display(head);

    pop(head);
    cout << "Elements in the priority queue after popping: ";
    display(head);
}

```

Output:

```
● PS C:\Second Year\3rd Semester\Advanced DSA using C++\Queue with Linked List> cd "c:\Second Year\3rd Semester\Advanced DSA using C++\Queue with Linked List\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ;  
Enter the number of elements to be inserted in the priority queue: 6  
Enter the elements: 345 3 56 761 4567 568  
Elements in the priority queue: 3 56 345 568 761 4567  
Elements in the priority queue after popping: 56 345 568 761 4567  
○ PS C:\Second Year\3rd Semester\Advanced DSA using C++\Queue with Linked List> █
```